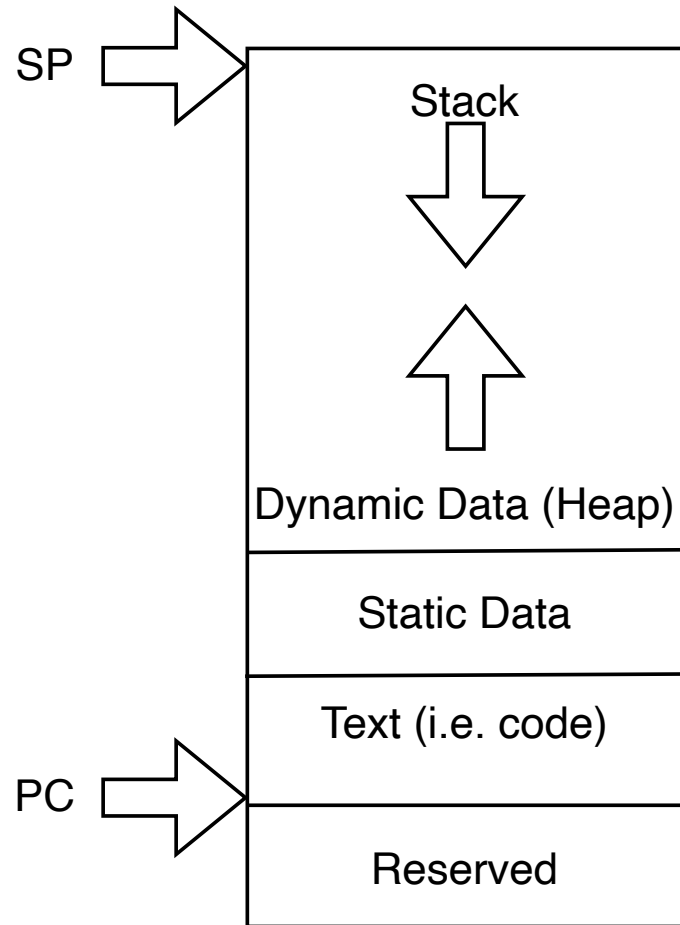


6. Jump and Subroutines

Memory Space



- This is the total memory available to a program (DRAM)
- *By convention*, code starts at `0000_0000_0040_0000hex` and grows up
- *By convention*, stack starts at `0000_003f_ffff_ffffhex` and grows down

Memory space

- Dynamic data is allocated by malloc(), etc
- Static data is global
- I/O is memory mapped to static data region
- In fact, any of these regions could be anywhere
- Code and stack addresses set by link loader
- NB If we don't follow these conventions, our code may not work with other people's libraries

Calling a Subroutine

1. Put parameters in a place where the subroutine can find them
2. Transfer control to the subroutine
3. Acquire the storage needed for the subroutine
4. Perform the task
5. Put the result in a place where the calling program can find it
6. Return control to the point of origin

Jump and link

- RISC-V has a jump and link instruction (`jal`) to jump to a location (step 2) and to save the return address (i.e. the PC)

```
jal x1, SubroutineAddress
```

- J (or UJ) format instruction (see next slide)
- 20 bits, bit 0 is implicitly 0.
- Can also do an unconditional branch

```
jal x0, Label
```

- `x0` is zero, therefore return address is thrown away

Core Instruction Formats

	31	25	24	20	19	15	14	12	11	7	6	0	
R	funct7			rs2		rs1		funct3		rd		opcode	
I	imm[11:0]					rs1		funct3		rd		opcode	
S	imm[11:5]			rs2		rs1		funct3		imm[4:0]		opcode	
B	imm[12 10:5]			rs2		rs1		funct3		imm[4:1 11]		opcode	
U	imm[31:12]									rd		opcode	
J	imm[20 10:1 11 19:12]									rd		opcode	

Regularity – Note how imm bits are shuffled to maintain regularity

Jump and link register

- To return from the subroutine (step 6), use jalr

```
jalr x0, 0(x1)
```

- PC now takes the value of x1
- Can also use the pseudo-instruction (alias)

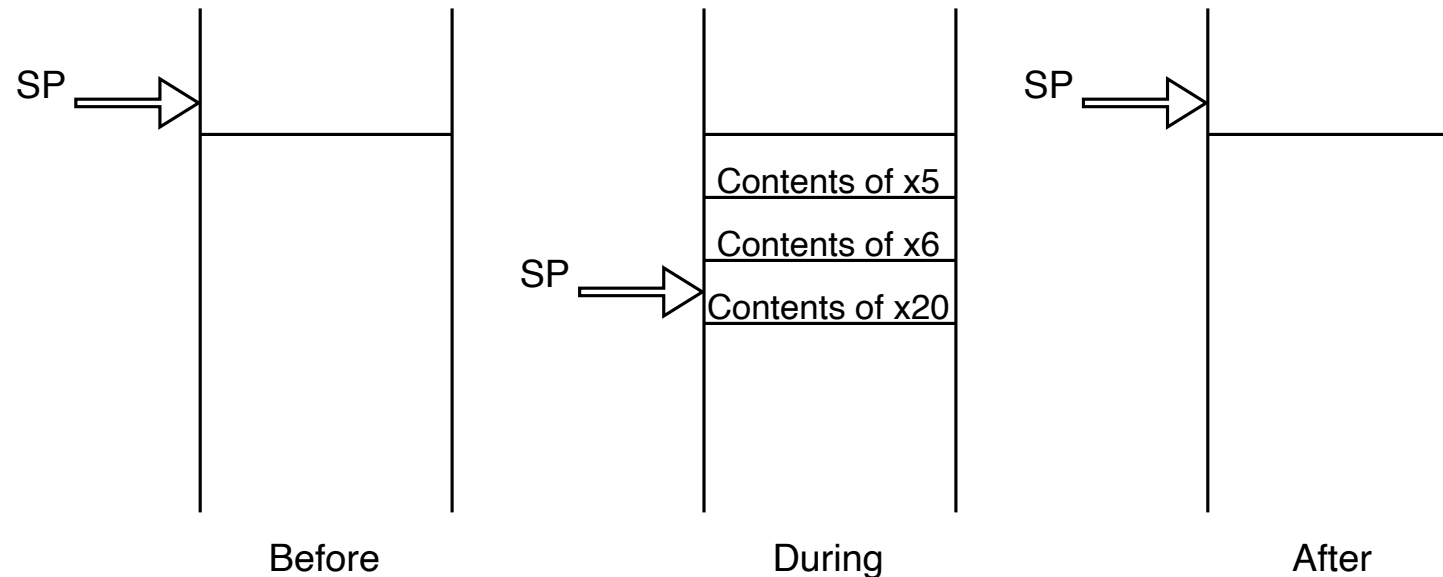
```
ret
```

- *By convention*, x1 is used for the return address and can also be referred to as ra
- None of this is hard-wired into the architecture. You don't have to follow convention, but you're on your own!

Stack

- In Step 3, we need to acquire storage – i.e. free up space in the instruction register
- Let's assume that we want to save the values of 3 registers so that we can use those values again on return, say, x5, x6, x20
- We *push* those values onto the stack
- We can then use those registers for a task, but don't save them
- When done, we *pop* the values off the stack back to the original registers and return

Stack operation



- Each register requires 4 bytes (4 addresses)
- 3 registers needs 12 bytes
- Stack pointer is decremented by 12

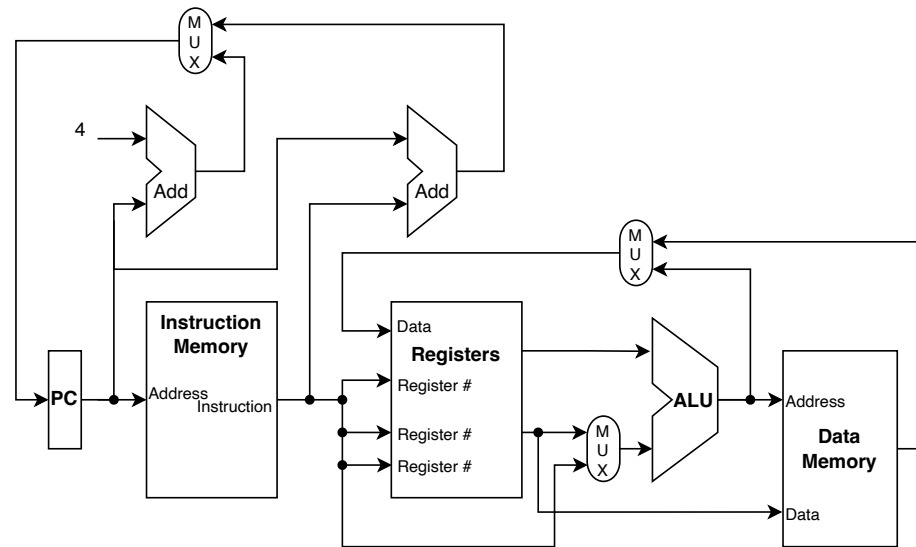
Stack operation

```
addi sp, sp, -12 // make space for 3 registers
sw  x5, 8(sp)    // save x5
sw  x6, 4(sp)    // save x6
sw  x20, 0(sp)   // save x20
```

- **Before return**

```
lw  x20, 0(sp)   // restore x20
lw  x6, 4(sp)    // restore x6
lw  x5, 8(sp)    // restore x5
addi sp, sp, 12  // restore stack pointer
```

Where's the stack pointer?



- *By convention, x2 is used as the stack pointer (sp)*

Register file convention

Register	Name	Use	Preserved	Saved by
x0	zero	constant 0	n.a.	n.a.
x1	ra	return address	yes	caller
x2	sp	stack pointer	yes	callee
x3	gp	global pointer	yes	-
x4	tp	thread pointer	yes	-
x5-x7	t0-t2	temporaries	no	caller
x8	so/fp	saved/frame pointer	yes	callee
x9	s1	saved	yes	callee
x10-x17	a0-a7	arguments/results	no	caller
x18-x27	s3-s11	saved	yes	callee
x28-x31	t3-t6	temporaries	no	caller

Wide immediate values

- Load Upper Immediate (U-type)
- Loads bits 31 down to 12

```
lui x19, 976 // 976decimal = 0000_0000_0011_1101_0000
```

- Register x19 becomes

```
0000_0000_0011_1101_0000_0000_0000_0000
```

- Can then do an `addi` to set the bottom 12 bits
- Thus we can create 32-bit immediate operands with 2 instructions

32-bit addresses

- Add Upper Immediate Program Counter (auipc)
- U-type
- Like `lui`, but adds 20 bit immediate to bits 31-12 of PC and saves in register

```
auipc x19, 976
```

- Useful for, e.g., dynamically linked libraries
- This covers the I instruction set, except for:
 - `fence`, `fence.i`, `ecall`, `ebreak`
 - 6 Control Status Register (CSR) instructions

For you to do

- I have put some SystemVerilog examples on Moodle.
 - Some of these need a bit more explanation... to follow
- Use these to practice SystemVerilog coding